



CV-Driven Window Selection for Multi-perspective Concept Drift Detection in Event Logs

Samira Khraiwesh (✉) , Dogukan Dagtekin, and Luise Pufahl 



CIT, Technical University of Munich,
Bildungscampus 2, 74076 Heilbronn, Germany
{samira.khraiwesh,luise.pufahl}@tum.de

Abstract. Process mining leverages event logs recorded in information systems to discover and analyze business processes. However, rapid technological and market changes often lead to evolving process behavior, which manifests as concept drift in event logs. While recent research has focused on detecting and explaining such drifts, many existing approaches focus on individual perspectives of process behavior, often emphasizing control-flow, while other perspectives receive less attention. In this paper, we propose a multi-perspective concept drift detection approach. The approach models process behavior as a time series and introduces a coefficient-of-variation-based strategy to automatically determine suitable window sizes for drift detection. This adaptive window selection enables more stable change-point detection across different process perspectives. We evaluate the approach against established baselines using synthetic and benchmark datasets. The results demonstrate competitive detection performance and show that the proposed method effectively identifies drifts across multiple perspectives of process behavior.

Keywords: Process Mining · Concept Drift · Multi-Perspective Analysis · Window selection

1 Introduction

Information systems have become essential infrastructures supporting modern organizations [2]. They record detailed information about the execution of business processes in the form of event logs [28], which constitute the primary input for process mining techniques. Process mining (PM) is a family of techniques to analyze, monitor, and improve business processes across various domains [28]. Core PM tasks include process discovery, conformance checking, and performance analysis [27]. Most of these techniques implicitly assume that process behavior remains relatively stable over time [4]. In reality, however, business processes operate in dynamic environments. Regulatory updates, technological changes, and organizational restructuring continuously reshape process behavior [4]. These changes are reflected in event logs and may alter control-flow,

temporal, or resource-related patterns and is referred to as *concept drift* [23]. Concept drift may be planned or unexpected and is recognized as a fundamental challenge in process mining [29]. Numerous approaches have been proposed to detect concept drift in event logs [4, 6, 30]. However, most focus on the control-flow perspective [23], while methods incorporating additional process perspectives remain limited [2, 23]. Yet, drift can also occur in temporal and resource perspectives, affecting the correctness of downstream PM applications, such as business process simulation and predictive monitoring.

Since drift detection commonly relies on comparing behavioral statistics over time, *window-based segmentation* plays a central role in many approaches [23]. Existing methods typically rely on either fixed or adaptive windowing strategies to segment the event log before comparing behavioral statistics. While fixed windows are simple and computationally efficient, their effectiveness strongly depends on the manually selected window size: small windows may be sensitive to noise, whereas large windows may smooth behavioral variations and delay or even miss drift points [13]. As a result, the effectiveness of many drift detection techniques strongly depends on the selected window size. Despite its critical influence on detection performance, the problem of automatically determining suitable window sizes from event logs has received limited attention in existing PM research [23]. Adaptive approaches mitigate this limitation by dynamically adjusting the window length [19], but are typically applied globally and do not account for the heterogeneous evolution of different process perspectives.

To address these limitations, we introduce a novel concept drift detection approach with two main contributions. First, we propose a multi-perspective drift detection framework (**CVDrift**) that analyzes control-flow behavior, activity durations, inter-arrival time, and resourceactivity relationships, enabling a more comprehensive and behavior-aware detection of process evolution. Second, we introduce a data-driven, component-specific window selection mechanism that determines suitable window sizes independently for different behavioral time series, enabling finer-grained and more robust drift detection. The remainder of the paper is organized as follows. Section 2 introduces the preliminaries, Sect. 3 reviews related work, Sect. 4 presents the proposed approach, Sect. 5 reports the experimental evaluation, and Sect. 6 concludes the paper.

2 Preliminaries

This section introduces the fundamental concepts for this paper, including event logs as the primary data source for concept drift detection, followed by the notion of concept drift and the process perspectives considered in this work.

Event Logs. In PM, an *event log* represents the recorded execution history. An event log L consists of a set of cases $L = \{\sigma_1, \sigma_2, \dots, \sigma_n\}$ where each case $\sigma = \langle e_1, e_2, \dots, e_k \rangle$ is an ordered sequence of events. An event can be represented as a tuple $e = (c, a, t_s, t_e, r)$ where c is the case identifier, a the activity, t_s the start timestamp, t_e the end timestamp, and r the resource performing the activity.

We define a projection function $\pi_x(e)$ that returns the value of attribute $x \in \{c, a, t_s, t_e, r\}$, where, e.g., $\pi_a(e)$ returns the activity of e . Additionally, we define the first-event projection as $first(\sigma) = e_1$. Accordingly, the start timestamp of a case is defined as $s(\sigma_i) = \pi_{t_s}(first(\sigma_i))$.

Concept Drift. Concept drift refers to changes in process behavior over time. Let B_t denote the process behavior at time t . A drift occurs when $B_{t_1} \neq B_{t_2}$ for $t_1 < t_2$. Drift may appear in different forms: *sudden drift*, where behavior changes abruptly at a point t_d ; *gradual drift*, where the transition occurs over an interval; and *recurring drift*, where a previously observed behavior reappears.

2.1 Process Perspectives

Control-Flow Perspective. The control-flow projection of a case is defined as

$CF(\sigma) = \langle \pi_a(e_1), \dots, \pi_a(e_n) \rangle$. *Routing behavior* captures the probability that activity b directly follows a is

$$P(a \rightarrow b) = \frac{|\{\sigma \in L, e_i, e_{i+1} \in \sigma \mid \pi_a(e_i) = a, \pi_a(e_{i+1}) = b\}|}{|\{\sigma \in L, e_i, e_{i+1} \in \sigma \mid \pi_a(e_i) = a\}|}$$

Temporal Perspective. The duration of an event e is defined as $dur(e) = \pi_{t_e}(e) - \pi_{t_s}(e)$. The *inter-arrival* time between two consecutive cases is defined as $t_{arr}(\sigma_i) = s(\sigma_i) - s(\sigma_{i-1})$.

Resource Perspective. The relationship between activities and resources is defined as the probability that activity a is performed by resource r , denoted by $P(a \rightarrow r)$, where

$$P(a \rightarrow r) = \frac{|\{\sigma \in L, e \in \sigma \mid \pi_a(e) = a, \pi_r(e) = r\}|}{|\{\sigma \in L, e \in \sigma \mid \pi_a(e) = a\}|}.$$

The extracted time series are organized as case-indexed time series ordered by case start times, enabling the detection of sudden changes in different process perspectives.

3 Related Work

Concept drift is a well-known challenge in process mining, and detecting such changes in event logs has attracted significant research attention [23]. Prior work mainly follows two directions: (i) detecting drifts and supporting their analysis, and (ii) developing online PM techniques for continuously evolving environments [23]. In this section, we focus on the first direction and review approaches that identify change points in event logs. Table 1 summarizes representative methods.

Existing approaches for change point detection in process mining can be broadly categorized into three groups: (i) approaches that propose new drift

Table 1. Overview of concept drift detection approaches in PM

Method	Perspective	Window Strategy	Window Scope
Bose et al. [4,5]	CF	Fixed	Global
Lin et al. [17]	CF	Fixed / Adaptive	Global
Zheng et al. [31]	CF	Fixed	Global
Maaradji et al. [18]	CF	Adaptive	Global
Martjushev et al. [20]	CF	Adaptive	Global
Neto et al. [21]	CF	Fixed	Global
Gallego-Fontenla et al. [12]	CF	Fixed	Global
Kraus and van der Aa [16]	CF	Fixed	Global
Brockhoff et al. [6]	CF, T	Fixed	Global
Adams et al. [2]	CF, T, R	Fixed	Global
Cao et al. [7]	CF, R	Fixed	Global
Klijn et al. [15]	CF, R	Fixed	Global

detection techniques, (ii) approaches that focus on adaptive window size strategies, and (iii) approaches that investigate multi-perspective drift detection.

The first category introduces techniques for detecting concept drift in event logs. Bose et al. [4,5] propose statistical features based on activity relations, namely Window Count and J-Measure, to detect changes across consecutive windows. Similarly, Lin et al. [17] analyze variations in directly-follows relations, while Zheng et al. [31] represent activity relations using Boolean matrices to identify behavioral changes over time. Other approaches rely on *alternative representations of process behavior*: Maaradji et al. [18] propose ProDrift, which applies statistical hypothesis testing on run distributions. Neto et al. [21] use transition matrices to capture control-flow behavior, Gallego-Fontenla et al. [12] combine process discovery and conformance checking, and Kraus and van der Aa [16] apply deep learning on image representations of event logs to detect drift locations and types. Despite this diversity, most approaches focus primarily on control-flow changes and rely on window-based log segmentation. Their performance therefore strongly depends on the chosen window size: small windows may be sensitive to noise, whereas large windows may smooth variations and delay or miss drift points.

To reduce the dependency on predefined window sizes, a second line of research investigates adaptive windowing strategies. Martjushev et al. [20] introduce an adaptive mechanism that dynamically adjusts the window size based on statistical evidence of change. Their method applies hypothesis testing to identify potential drift regions and progressively refines the analyzed window boundaries in order to better localize drift points. Although such strategies alleviate the need for a single fixed window size, they rely on reactive drift-detection mechanisms that dynamically update a single window over a data stream, which may overlook the heterogeneous evolution of different process perspectives.

A third line of research investigates concept drift detection from multiple process perspectives, recognizing that process evolution may occur not only in control-flow but also in other behavioral dimensions. For example, Brockhoff et al. [6] incorporate temporal characteristics using an Earth Mover’s Distance (EMD)-based approach, while Adams et al. [2] propose an explainable framework analyzing multiple process measures. Cao et al. [7] introduce the MDD approach, integrating perspectives such as activities, resources, and transitions using graph-based representations, and [15] extends drift detection to the organizational perspective by incorporating actor-related information. Although these approaches demonstrate the benefits of multi-perspective analysis, they typically rely on a single global window configuration to segment the event log. However, control-flow relations, temporal patterns, and resource behavior may evolve at different temporal scales, making a uniform window configuration insufficient to capture the heterogeneous dynamics of process evolution.

We propose a data-driven mechanism that determines window sizes per perspective based on the stability of the coefficient of variation (CV), enabling robust multi-perspective drift detection.

4 Approach

To detect drifts across the control flow, temporal and resource perspectives introduced in Sect. 2, the proposed method employs a detection pipeline consisting of three main stages: (i) event log preparation and initial time-series construction, (ii) CV-based window selection, and (iii) change-point detection.

4.1 Event Log Preparation and Initial Time-Series Construction

The proposed pipeline transforms an event log L into a set of case-indexed time series capturing different process perspectives. Let $L = \{\sigma_1, \dots, \sigma_n\}$ denote the set of cases, ordered by their start timestamps $s(\sigma_i)$. Within each case, events are ordered also according to their start timestamps. This ordering induces a temporal index over cases, enabling their sequential analysis.

For each case σ_i , a single numerical observation is derived per perspective. This results in a collection of aligned, case-indexed time series, where each observation is associated with the start timestamp $s(\sigma_i)$ of its corresponding case. While the previous definitions capture global log-level characteristics, the following constructs derive case-indexed time series that approximate these measures locally over time and serve as input for the subsequent drift detection procedure.

Temporal Perspective. For each case index $i \in \{1, \dots, n\}$, an activity-specific time series for the *duration* of activity a is defined as $t_a(i) = \text{median}\{dur(e) \mid e \in \sigma_i, \pi_a(e) = a\}$. If activity $\pi_a(e) = a$ appears multiple times within a case, the median duration is used to reduce the influence of extreme values. Cases in which the activity does not occur are assigned missing values. An example event log and derived initial time series is shown in Fig. 1.

The *inter-arrival* time series is defined as $t_{\text{arr}}(i) = s(\sigma_i) - s(\sigma_{i-1})$. To avoid distortions caused by overnight gaps or system downtime, arrivals spanning different calendar days or exceeding a predefined maximum gap are excluded.

Case ID	Activity	Start	End	Resource
i	a	t_s	t_e	r
1	A	10:00:00	10:05:00	R1
1		...		
2	A	10:10:00	10:17:00	R1
2		...		
n	A	12:00:00	12:10:00	R1

$$t_A = \{t_A(1), t_A(2), \dots, t_A(n)\}$$

$$t_A = \{5, 7, \dots, 10\}$$

Fig. 1. Example event log and derived initial time series for activities A

Control-Flow Perspective. Let $\mathcal{P} \subseteq \mathcal{A} \times \mathcal{A}$ denote the set of activity pairs observed in the directly-follows relation. For each pair $(A, B) \in \mathcal{P}$ and each case index $i \in \{1, \dots, n\}$, the routing time series is defined as

$$t_{A,B}(i) = \frac{|\{e_j, e_{j+1} \in \sigma_i \mid \pi_a(e_j) = A, \pi_a(e_{j+1}) = B\}|}{|\{e_j, e_{j+1} \in \sigma_i \mid \pi_a(e_j) = A\}|}.$$

Cases in which activity a does not occur are assigned missing values.

Resource Perspective. For each activity–resource pair (A, R) , the corresponding time series captures the proportion of executions in which resource R performs activity A :

$$t_{A,R}(i) = \frac{|\{e \in \sigma_i \mid \pi_a(e) = A, \pi_r(e) = R\}|}{|\{e \in \sigma_i \mid \pi_a(e) = A\}|},$$

4.2 CV-Based Window Selection

The case-indexed time series constructed in the previous step serve as the input for the window-selection stage. Let $t = \{t(1), \dots, t(n)\}$ denote one of the derived time series. Given a set of candidate window sizes $W = \{w_1, \dots, w_m\}$, a rolling transformation is applied to t . For a window size $w \in W$, a rolling window of w consecutive cases is moved across the series to produce a smoothed sequence

$$r_w(t) = \{s_w(1), \dots, s_w(n - w + 1)\},$$

where $s_w(j)$ denotes a summary statistic computed over the window $\{t(j), \dots, t(j+w-1)\}$. The choice of the statistic depends on the process perspective. For temporal series (activity duration and inter-arrival time), the median is used to provide robustness against extreme values, whereas for control-flow and resource activity–resource relationships perspective, the mean is used to estimate

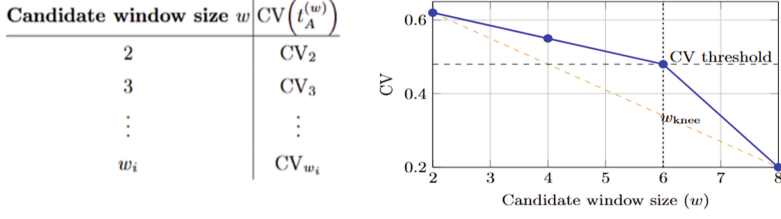


Fig. 2. Knee-point detection based on CV values

the relative frequency of the observed relation. For each candidate window size w , the stability of the resulting rolling series is evaluated using the coefficient of variation (CV) [10], defined as

$$CV_w = \frac{\sigma(r_w(t))}{|\mu(r_w(t))|},$$

where $\sigma(\cdot)$ and $\mu(\cdot)$ denote the standard deviation and mean of the rolling values, respectively. The CV measures the relative variability of the smoothed signal: smaller values indicate a more stable series, while larger values reflect stronger fluctuations. The candidate window sizes and their corresponding CV values form a curve $\{(w_i, CV_{w_i})\}_{i=1}^m$ (see Fig. 2). Inspired by geometric knee-point detection methods [24], the knee of the normalized CV curve is identified as the point with the maximum perpendicular distance from the line connecting the first and last points as shown in Fig. 2. Let the CV curve consist of points $P_i = (w_i, CV_{w_i})$ for $i = 1, \dots, m$. After normalizing both axes, the perpendicular distance of each point to the line connecting the first and last points is computed as

$$d_i = \frac{|(B_x - A_x)(P_{i,y} - A_y) - (B_y - A_y)(P_{i,x} - A_x)|}{\sqrt{(B_x - A_x)^2 + (B_y - A_y)^2}} \quad (1)$$

where $A = (w'_1, CV'_{w_1})$ and $B = (w'_m, CV'_{w_m})$ denote the first and last points of the normalized curve. The knee point is then selected as

$$k = \arg \max_i d_i.$$

The CV value at this point defines a threshold, and the smallest candidate window whose CV does not exceed this value is selected. Each perspective, and more specifically each component, is treated independently; for example, in the activity duration perspective, each activity is assigned its own window size, which is used to construct the final smoothed series for subsequent change-point detection. The procedure is summarized in Algorithm 1.

The window size can be determined using two strategies. The **CVPer-Pair** strategy assigns a separate window size to each activity, routing relation, or resource relationship based on its behavior. In contrast, the **ModeWindow**

Algorithm 1: CV-Based Window Selection

Input: Time-series set T , candidate window sizes $W = \{w_1, \dots, w_m\}$ **Output:** Selected window size w_t^* for each series $t \in T$ **foreach** $t \in T$ **do** Initialize $CV_scores \leftarrow \emptyset$; **foreach** $w \in W$ **do** Compute rolling series $r_w(t)$ using window size w ; Compute coefficient of variation $CV_w = \sigma(r_w(t)) / |\mu(r_w(t))|$; Store (w, CV_w) in CV_scores ; Construct CV curve $\{(w_i, CV_{w_i})\}$; Identify knee point k using the maximum perpendicular distance criterion (Eq. 1); Select w_t^* as the smallest w such that $CV_w \leq CV_k$;

strategy selects a single window size per perspective, defined as the most frequently occurring window size across all pairs within that perspective. This window is specific for a process perspective rather than global to the entire event log.

4.3 Change-Point Detection

After selecting the window size, a rolling statistic is computed over consecutive cases for each series. Change-point detection is then performed on the resulting rolling series using the Pruned Exact Linear Time (PELT) algorithm [14]. To improve robustness, each series is analyzed at multiple window resolutions, nearby detections within a series are merged, and very small shifts are removed using a minimum effect-size threshold. Nearby detections across related series are clustered to identify consistent changes. Although the window size is selected automatically, the PELT algorithm still requires a penalty parameter that influences the number of detected change points. The proposed pipeline reduces manual effort in window-size selection, but it is not fully parameter-free. The influence of the PELT penalty and the effect-size threshold is examined in the sensitivity analyses in Sect. 5.

5 Evaluation

This section presents two experiments evaluating the proposed approach: Experiment 1 focuses on the temporal perspective, while Experiment 2 targets the control-flow perspective. Similar to prior work [7], experiments are conducted separately, as datasets typically contain drifts affecting specific process perspectives. The datasets, implementation, and results are publicly available in our repository. The proposed framework supports several process perspectives, including control-flow, temporal, and resource-related behavior. In the evaluation, we focus on the temporal and control-flow perspectives because suitable benchmark settings and ground-truth drift points are available for these perspectives. The resource perspective is not included in the evaluation due to the lack of publicly available datasets with labeled resource-perspective drifts. We

therefore interpret the evaluation as evidence for selected perspectives rather than as a complete validation of all possible process perspectives.

5.1 Experiment 1: Temporal Perspective

In this experiment, we evaluate drift in activity durations¹. As no publicly available datasets contain controlled temporal drifts with ground truth, we generated synthetic datasets using a custom script²

Datasets. We constructed a synthetic event log corpus using PM4Py [1] based on the process model. The corpus consists of 72 event logs, including 18 clean logs and 54 noisy logs, covering six drift magnitudes, three log sizes, and four noise levels (including 0%). The clean logs were generated using activity durations following a normal distribution with a 20% standard deviation, while inter-activity waiting times range from one to three minutes within working hours (08:0015:00). This configuration reflects typical real-world process behavior. To introduce duration drift, we selected activities A, B, and D – spanning different branches of the process model to ensure that the drift affects multiple execution paths rather than a single bottleneck. At two predefined change points, located at 37% and 75% of the total log length, the durations of these activities are increased by a fixed amount. The asymmetric placement allows evaluation of both early and late drift detection. Six drift magnitudes were applied (10, 20, 30, 40, 50, and 60 min), ranging from subtle shifts comparable to the natural variance to large disruptions. Each configuration was generated at three log sizes (100, 1,000, and 3,000 cases), yielding 18 clean logs. To evaluate robustness to noise, we applied the FLAWD noise injection framework [11] to each clean log at three noise ratios (5%, 10%, and 20%). The noise injection targeted two types of anomalies: (i) random character-level typographical errors in activity names, simulating manual data-entry mistakes, and (ii) stochastic shifts in event timestamps, mimicking real-world temporal anomalies such as system delays or late data entries. These noise levels represent low, moderate, and high data quality degradation commonly encountered in practice, resulting in 54 noisy logs.

Baseline. We compare our approach with the **OCPA** technique [2], which constructs time series from process measures and applies the PELT change-point detection algorithm. The method relies on a fixed global time window. We used the authors’ original implementation and developed a wrapper to evaluate it on our benchmark datasets. Since the code requires Object-Centric Event Logs (OCEL), standard XES logs were converted into a pseudo-OCEL format by mapping each case identifier to a single object type, following the authors’ description. We used the parameter configuration provided in the authors’ drift detection example, where a fixed window size of 7 d is applied, as it was reported

¹ Drift detection for arrival times is also implemented in our repository.

² Available at <https://zenodo.org/records/20497480>.

to yield the best results. Although [7] aims to support multiple perspectives, the available implementation covers only control flow and therefore cannot serve as a baseline for the temporal perspective in this study. For **CVDrift**, 15 candidate window sizes ranging from 15 to 5000 cases were considered for the window selection step. Change-point detection was performed using the PELT algorithm with a penalty parameter $\lambda = 3$ and a minimum mean difference threshold of 0.00. The impact of these parameters is examined in the sensitivity analysis presented later.

Evaluation Metrics. Detection performance is evaluated using precision, recall, and F1-score by matching detected drift points with ground-truth drifts. To account for detection delays, we apply a tolerance window. A detected drift within this tolerance is counted as a true positive. OCPA and CVDrift operate in different indexing spaces: CVDrift outputs case-indexed changepoints, whereas OCPA detects changes on fixed-duration time windows and maps them to median case IDs, which introduces localization uncertainty. Accordingly, evaluation is performed using tolerance-based matching. We use 10% of the dataset size as the primary tolerance window, while CVDrift also shows strong performance at stricter tolerances.³

Table 2. Results of P (precision), R (recall), and F1-score by Drift Magnitude

Technique	10 min			20 min			30 min		
	P	R	F1	P	R	F1	P	R	F1
CVDrift	1.000	0.944	0.971	0.810	0.773	0.791	0.857	0.750	0.800
OCPA	0.833	0.555	0.667	0.875	0.636	0.737	0.842	0.667	0.744
Technique	40 min			50 min			60 min		
	P	R	F1	P	R	F1	P	R	F1
CVDrift	0.850	0.708	0.773	0.875	0.875	0.875	0.870	0.833	0.851
OCPA	0.842	0.667	0.744	0.254	0.625	0.361	0.068	0.750	0.125

Tables 2 and 3 present the precision, recall, and F1-score comparison between our approach and OCPA by drift magnitude and noise level. The results in Table 2 show that CVDrift achieves consistently high performance across all drift magnitudes, with F1 scores ranging from 0.800 to 1.000. This stability is due to its window selection mechanism, which adapts to the statistical properties of each time series. In contrast, OCPA achieves competitive precision for

³ Our algorithm achieves an F1 score of 0.88 (5%, 3%, and 2% tolerances). Full results are available in the repository.

small-to-moderate drifts; its precision drops significantly at larger magnitudes, indicating over-segmentation. This aligns with prior findings [3], where smaller drifts require larger observation windows, while larger drifts can be detected with fewer observations. CVDrift implicitly captures this relationship through its data-driven window selection. As shown in Table 3, CVDrift demonstrates robust performance across noise levels, with only a gradual decline as noise increases, and consistently outperforms the baseline (OCPA). We have tested our approach used for detecting the concept drift using *CVPerPair* and *Mod-eWindow* strategies and both provides the same results

Table 3. Results of P (precision), R (recall), and F1-score by Noise Level

Technique	Clean			5%			10%			20%		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
CVDrift	1.000	0.917	0.957	0.839	0.813	0.825	0.839	0.765	0.800	0.807	0.735	0.769
OCPA	0.452	0.666	0.539	0.166	0.625	0.263	0.221	0.677	0.333	0.198	0.647	0.303

Sensitivity Analysis. To examine the impact of the PELT penalty scale and the effect-size filter, we conducted a sensitivity analysis over a grid of penalty scales ($\beta \in \{2, 3, 5, 7\}$) and minimum effect-size thresholds ($d_{\min} \in \{0.00, 0.10, 0.15, 0.30\}$), resulting in 16 configurations. These parameters were selected because they directly control the sensitivity of change-point detection, determining how many change points are detected and which of them are retained after filtering. The aggregated results are shown in Fig. 3. Overall, the results indicate that detection performance is primarily influenced by the penalty scale, while the effect-size filter provides limited additional benefit. The best configuration ($\beta = 3$, $d_{\min} = 0.00$) achieves an F1 score of 0.925 with a high recall (0.993). This indicates that no effect-size filtering is required, as the algorithm already reliably detects relevant change points. Lower penalties increase false positives, whereas higher penalties suppress true drift points. The tested penalty values follow BIC-type recommendations [26], and the effect-size thresholds follow standard conventions [9]. The full factorial grid assesses both parameters and their interaction.

5.2 Experiment 2: Control-Flow Perspective

In this section, we evaluate our approach’s ability to detect change points in changes in routing behavior from a control-flow perspective. The performance of our approach is compared with existing baselines using synthetic datasets.

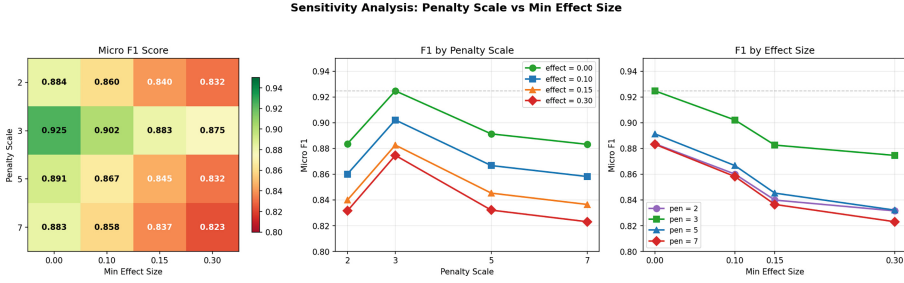


Fig. 3. Sensitivity results across penalty scales and effect-size thresholds.

Datasets. We use a publicly available benchmark dataset⁴ previously employed for evaluating concept drift detection methods [1]. The dataset consists of synthetic event logs from three studies [5, 8, 22], with about 1700 traces on average and one to four annotated change points. To enable direct comparison, we applied our approach to the same logs and compared the results with those reported in the benchmark.

Baselines. We compare our method against seven concept drift detection techniques that were evaluated in the benchmark study of Adams et al. [1]: **Bose/J** [5], **Adwin/J** [20], **ProGraphs** [25], **ProDrift** [19], **RINV** [31], **EMD**, [6], and **LCDD** [17]. For **CVDrift**, 15 candidate window sizes ranging from 15 to 5000 cases were considered for the window selection step. Change-point detection was performed using the PELT algorithm with a penalty parameter $\lambda = 7$ and a minimum mean difference threshold of 0.15 and using *ModeWindow* strategy. The impact of these parameters, including the strategy, is further examined in the sensitivity analysis presented later.

Evaluation Metrics. To evaluate the performance of the proposed approach, we use the evaluation metrics from the benchmark study by Adams et al. [1], focusing on detection accuracy based on a lag window corresponding to the equivalent time of 200 process executions. In addition, we consider versatility across different drift types since this dimension is influenced by the chosen window size, as different change patterns manifest at different temporal scales.

Results. Figure 4 shows the comparison of the evaluated approaches in terms of F1 score on the noiseless datasets. The results indicate that CVDrift achieves competitive performance compared to the other approaches. In particular, it performs comparably to methods relying on fixed window sizes, while outperforming several approaches that employ adaptive windowing strategies.

Figure 5 presents performance across varying noise levels using non-fitting events (datasets from [22]). CVDrift maintains stable performance across all

⁴ <https://github.com/cpitsch/cdrift-evaluation>.

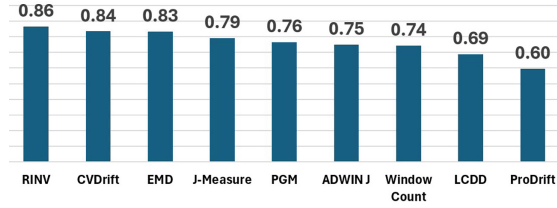


Fig. 4. Average F1-score comparison of drift detection algorithms across the benchmark datasets.

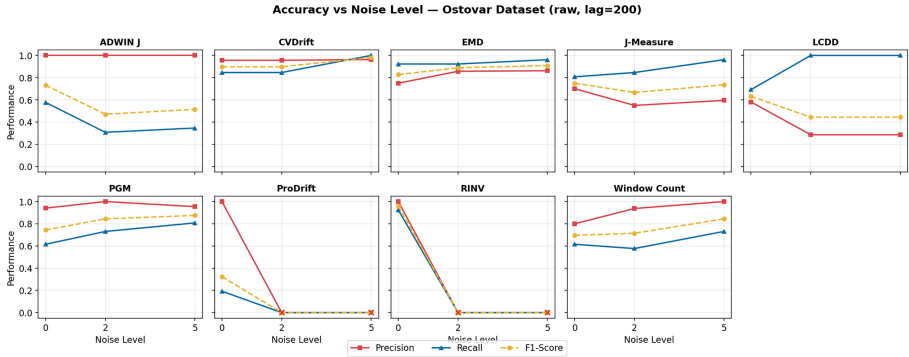


Fig. 5. Performance comparison of drift detection algorithms across noise levels.

noise levels and shows more consistent behavior than adaptive window-based approaches such as ADWIN, PGM, and LCDD, which exhibit noticeable fluctuations. Its performance is also comparable to fixed-window methods such as Window Count and EMD, while maintaining a balanced trade-off between precision and recall. Figure 6 illustrates the versatility of the evaluated approaches across different change patterns. Most window-based methods exhibit uneven performance, reflecting the bias-variance trade-off inherent to fixed observation windows, where a window size suitable for one change pattern may be suboptimal for another. Overall, CVDrift achieves competitive versatility ($V = 0.83$), with performance comparable to EMD and RINV and superior to methods that rely on adaptive window strategies. However, the approach shows weaker performance for frequency (fr) and conditional removal (cre) patterns. For frequency changes, this may be explained by the fact that CVDrift models routing behavior using probabilities rather than absolute frequencies, making subtle frequency variations less visible in the derived signal. In the case of CRE, the change affects only a subset of traces where an activity is conditionally skipped, while the overall process structure remains largely unchanged, resulting in a relatively weak signal that is harder to detect.

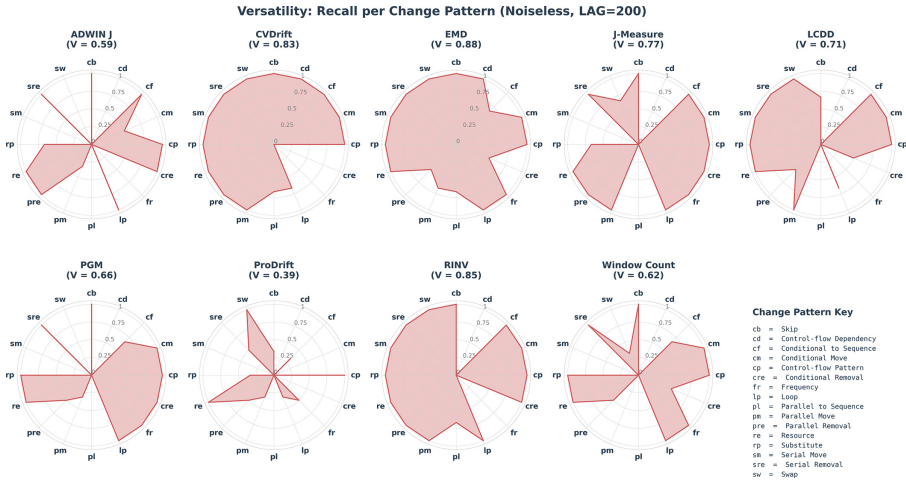


Fig. 6. Versatility: Recall per Change Pattern

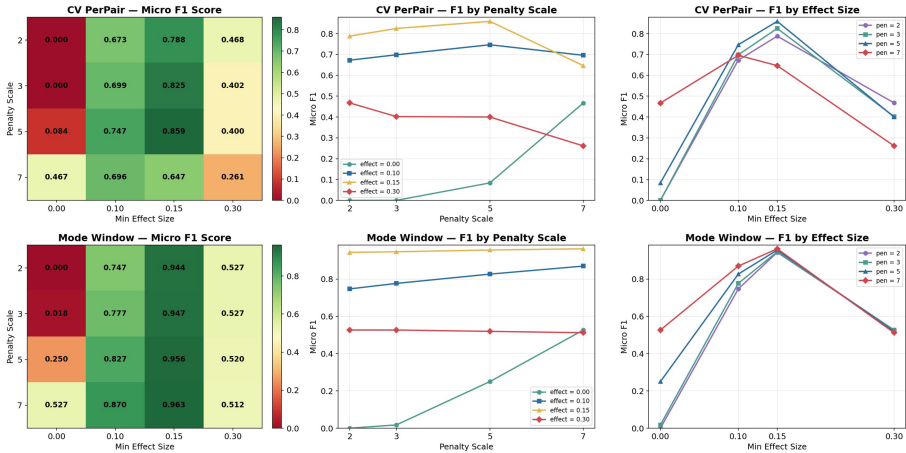


Fig. 7. Sensitivity results across penalty scales, effect-size thresholds, and strategies.

Sensitivity Analysis To examine the impact of the PELT penalty scale, the effect-size filter, and the window strategies, we conducted a sensitivity analysis. Specifically, we evaluated penalty scales of 2, 3, 5, and 7, and effect-size thresholds of 0.00, 0.10, 0.15, and 0.30, resulting in 16 configurations. These configurations were tested using two window strategies (*CVPerPair* and *ModeWindow*), yielding a total of 32 configurations. This sensitivity analysis is done on Ostovar dataset [22]. The results shown in Fig. 7 indicate that the minimum effect size has the strongest influence on the detection performance. Very small thresholds (e.g., 0.00) lead to poor results due to the detection of minor fluctuations, while large thresholds (e.g., 0.30) may suppress relevant changes. In

contrast, moderate values between 0.10 and 0.15 consistently yield the best performance. The penalty parameter shows a smaller impact on the results, with comparable performance observed across several values. Furthermore, the *ModeWindow* strategy produces more consistent results compared to the per-pair approach, supporting its use for structural perspectives.

6 Discussion and Conclusion

The experimental results show that the proposed CVDrift approach achieves competitive and often superior performance in detecting concept drift across multiple process perspectives. Although it relies on the widely used PELT change-point detection algorithm, its effectiveness stems from generating different time series for multiple process perspectives and applying a data-driven window selection strategy based on the characteristics of each time series. These results highlight that drift detection performance depends not only on the underlying algorithm but also strongly on how the observation window is defined. Another observation is that the proposed approach performs very well for the temporal perspective (activity durations) without requiring additional effect-size filtering, indicating that the detected changes are sufficiently strong and reliable. In contrast, applying the approach to the control-flow perspective requires a minimum effect-size threshold to achieve optimal results. This difference can be attributed to the nature of the underlying time series. Temporal time series are typically unbounded and exhibit higher variability, allowing drift to manifest as pronounced absolute changes. In contrast, control-flow time series are bounded and often represented as probabilities, which compress their magnitude and make them less distinguishable from noise. The same reasoning also explains why the *ModeWindow* strategy performs better than the *CVPerPair* strategy for the control-flow perspective. A key contribution of this study is the introduction of a data-driven window selection strategy that eliminates the need for manual window size specification. This represents an important step toward more automated drift detection in process mining settings, where event logs vary significantly in size, granularity, and the nature of underlying process changes. While the window-selection component is entirely data-driven, the PELT algorithm still requires a penalty parameter. The sensitivity analyses indicate that the penalty parameter has a smaller impact on detection performance than other configuration choices, with comparable performance observed across several penalty values. In contrast, inappropriate window sizes may directly lead to missed or delayed drift detections, as discussed in Sect. 1. Furthermore, the knee-point strategy selects the *smallest* window achieving sufficient stability rather than the globally smoothest window, thereby preserving sensitivity to genuine drift signals and reducing the risk of smoothing away meaningful behavioral changes. The penalty parameter relates solely to the change-point detection algorithm and could be replaced by alternative algorithms or adaptive parameter selection strategies in future work. This separation of concerns—automated window selection combined with configurable change-point detection—provides

a foundation for further automation. Future research could integrate the proposed strategy with recent approaches that require little or no manual configuration beyond window definition, such as the approach proposed by Kraus et al. [16]. The current evaluation focuses on sudden drifts, where ground truth is well-defined and results can be compared objectively across methods. Extending CVDrift to gradual, recurring, and overlapping drifts remains an important direction for future work. Similarly, while the temporal evaluation relies on synthetic datasets, the use of logs with known drift locations and magnitudes enables rigorous and objective assessment. Additional validation on real-world event logs is required to assess the approach under more complex and heterogeneous process environments. A significant research opportunity concerns evaluation infrastructure. Currently, publicly available datasets and generators provide limited coverage of multi-perspective concept drifts with labeled ground truth. Developing comprehensive multi-perspective drift generators and standardized benchmarking frameworks, comparable to the benchmark by Adams et al. [1] but covering multiple perspectives, would substantially improve the reproducibility and comparability of future research. The current behavioral representations also point toward concrete and actionable extensions. For the control-flow perspective, CVDrift relies on probability-based directly-follows representations. This representation is intentionally simple and interpretable, and the analysis in Sect. 5 characterizes where it reaches its limits—particularly for frequency-related changes and conditional removals that affect only a subset of traces. Incorporating richer representations, such as eventually-follows relations or explicit decision points, could directly address these patterns. Similarly, the current resource perspective models activity–resource relationships under the assumption that a single resource is associated with each activity execution. Extending this representation to incorporate role-based behavior, workload distribution, or collaborative resource patterns would improve the framework’s ability to capture more complex organizational changes. Finally, the perspective-specific window selection opens interesting optimization opportunities. CVDrift evaluates multiple candidate window sizes for each generated time series before applying change-point detection, which enables component-specific and perspective-specific adaptation. However, this also introduces additional computational overhead compared to approaches that rely on a single predefined window size. Investigating efficient pruning strategies, scalability characteristics, and parallelization approaches therefore represents an important direction for future work. Overall, the results demonstrate that perspective-specific, data-driven window selection is a promising direction for improving concept drift detection in event logs. By reducing dependence on manually selected window sizes and supporting multiple process perspectives, CVDrift contributes toward more robust and practical drift detection in dynamic business process environments.

References

1. Adams, J.N., Pitsch, C., Brockhoff, T., Van Der Aalst, W.M.: An experimental evaluation of process concept drift detection. *Proc. VLDB Endowment* **16**(8), 1856–1869 (2023)
2. Adams, J.N., van Zelst, S.J., Rose, T., van der Aalst, W.M.: Explainable concept drift in process mining. *Inf. Syst.* **114**, 102177 (2023)
3. Bifet, A., Gavalda, R.: Learning from time-changing data with adaptive windowing. In: *Proceedings of the 2007 SIAM International Conference on Data Mining*, pp. 443–448. SIAM (2007)
4. Bose, R.J.C., van der Aalst, W.M., Žliobaitė, I., Pechenizkiy, M.: Handling concept drift in process mining. In: *International Conference on Advanced Information Systems Engineering*, pp. 391–405. Springer (2011)
5. Bose, R.J.C., Van Der Aalst, W.M., Žliobaitė, I., Pechenizkiy, M.: Dealing with concept drifts in process mining. *IEEE Trans. Neural Netw. Learn. Syst.* **25**(1), 154–171 (2013)
6. Brockhoff, T., Uysal, M.S., van der Aalst, W.M.: Time-aware concept drift detection using the earth mover’s distance. In: *2020 2nd International Conference on Process Mining (ICPM)*, pp. 33–40. IEEE (2020)
7. Cao, Y., Lin, L., Di, Y., Li, X., Chen, W.: MDD: process drift detection in event logs integrating multiple perspectives. In: *2024 IEEE International Conference on Web Services (ICWS)*, pp. 1125–1134. IEEE (2024)
8. Ceravolo, P., Tavares, G.M., Junior, S.B., Damiani, E.: Evaluation goals for online process mining: a concept drift perspective. *IEEE Trans. Serv. Comput.* **15**(4), 2473–2489 (2020)
9. Cohen, J.: *Statistical power analysis for the behavioral sciences*. Routledge (2013)
10. Colin Koeniguer, E., Nicolas, J.M.: Change detection based on the coefficient of variation in SAR time-series of urban areas. *Remote Sens.* **12**(13), 2089 (2020)
11. Comuzzi, M., Ko, J., Maggi, F.: A language to model and simulate data quality issues in process mining. *ACM J. Data Inf. Qual.* **17**(2), 1–36 (2025)
12. Gallego-Fontenla, V., Vidal, J.C., Lama, M.: A conformance checking-based approach for sudden drift detection in business processes. *IEEE Trans. Serv. Comput.* **16**(1), 13–26 (2021)
13. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. *ACM Comput. Surv. (CSUR)* **46**(4), 1–37 (2014)
14. Killick, R., Fearnhead, P., Eckley, I.A.: Optimal detection of changepoints with a linear computational cost. *J. Am. Stat. Assoc.* **107**(500), 1590–1598 (2012)
15. Klijn, E.L., Mannhardt, F., Fahland, D.: Multi-perspective concept drift detection: including the actor perspective. In: *International Conference on Advanced Information Systems Engineering*, pp. 141–157. Springer (2024)
16. Kraus, A., Van der Aa, H.: Machine learning-based detection of concept drift in business processes. *Process Sci.* **2**(1), 5 (2025)
17. Lin, L., Wen, L., Lin, L., Pei, J., Yang, H.: LCDD: detecting business process drifts based on local completeness. *IEEE Trans. Serv. Comput.* **15**(4), 2086–2099 (2020)
18. Maaradji, A., Dumas, M., La Rosa, M., Ostovar, A.: Fast and accurate business process drift detection. In: *International Conference on Business Process Management*, pp. 406–422. Springer (2015)
19. Maaradji, A., Dumas, M., La Rosa, M., Ostovar, A.: Detecting sudden and gradual drifts in business processes from execution traces. *IEEE Trans. Knowl. Data Eng.* **29**(10), 2140–2154 (2017)

20. Martjushev, J., Bose, R.J.C., Van Der Aalst, W.M.: Change point detection and dealing with gradual and multi-order dynamics in process mining. In: International Conference on Business Informatics Research, pp. 161–178. Springer (2015)
21. Meira Neto, A.C., de Sousa, R.G., Fantinato, M., Peres, S.M.: Revisiting the transition matrix-based concept drift approach: Improving the detection task reliability through additional experimentation. *SN Comput. Sci.* **5**(1), 188 (2024)
22. Ostovar, A., Maaradji, A., La Rosa, M., ter Hofstede, A.H., van Dongen, B.F.: Detecting drift from event streams of unpredictable business processes. In: International Conference on Conceptual Modeling, pp. 330–346. Springer (2016)
23. Sato, D.M.V., De Freitas, S.C., Barddal, J.P., Scalabrin, E.E.: A survey on concept drift in process mining. *ACM Comput. Surv. (CSUR)* **54**(9), 1–38 (2021)
24. Satopaa, V., Albrecht, J., Irwin, D., Raghavan, B.: Finding a” kneedle” in a haystack: Detecting knee points in system behavior. In: 2011 31st International Conference on Distributed Computing Systems Workshops, pp. 166–171. IEEE (2011)
25. Seeliger, A., Nolle, T., Mühlhäuser, M.: Detecting concept drift in processes using graph metrics on process graphs. In: Proceedings of the 9th Conference on Subject-Oriented Business Process Management, pp. 1–10. (2017)
26. Truong, C., Oudre, L., Vayatis, N.: Selective review of offline change point detection methods. *Signal Process* **167**, 107299 (2020)
27. Van Der Aalst, W.: Process mining: overview and opportunities. *ACM Trans. Manag. Inf. Syst. (TMIS)* **3**(2), 1–17 (2012)
28. Van Der Aalst, W.: Data science in action. In: Process mining: data science in action, pp. 3–23. Springer (2016)
29. Van Der Aalst, W., et al.: Process mining manifesto. In: International Conference on Business Process Management, pp. 169–194. Springer (2011)
30. Yeshchenko, A., Di Ciccio, C., Mendling, J., Polyvyanyy, A.: Visual drift detection for event sequence data of business processes. *IEEE Trans. Visual Comput. Graph.* **28**(8), 3050–3068 (2021)
31. Zheng, C., Wen, L., Wang, J.: Detecting process concept drifts from event logs. In: OTM Confederated International Conferences” On the Move to Meaningful Internet Systems”, pp. 524–542. Springer (2017)